

流水线并行（PP）深度笔记 —— 训练侧原理 / 经典调度 / torchtitan 实现

面向：attention residual + torchtitan 工作 / ML infra 面试级广度 + 实现级深度
视角：distributed systems + networking 类比（SDN / RDMA / NCCL 已熟悉）

1. PP 本质：通信模式决定一切

PP 和 TP/DP 的根本差异在**通信模式**，不在"切了什么"：

并行维度	通信模式	频率	单次量	网络要求
DP	all-reduce（全域）	每 step 末 1 次	$O(\text{参数量})$	高带宽，跨节点可
TP	all-reduce / reduce-scatter	每个 matmul	$O(\text{activation})$	NVLink 级，必须 intra-node
PP	point-to-point（邻居）	每 stage 边界	$O(\text{activation})$	IB/RoCE 足够

用你熟悉的网络术语：DP 像周期性 multicast，TP 像 flooding，PP 是纯 unicast，只有 rank $i \rightarrow \text{rank } i+1$ 的邻居关系。这就是 PP 为什么天然适合 inter-node（IB/RoCE）、而 TP 必须留在 NVLink 域内的根本原因。

PP 所有工程问题围绕一件事：**pipeline 永远有 bubble，怎么把它挤到最小**。下面所有算法都是在 1F1B 的基础上减 bubble。

2. 经典调度算法

2.1 GPipe（naive microbatching）

F 和 B 两个阶段**严格分开**：所有 microbatch 先依次走完正向，再依次反向。

- Bubble fraction： $\frac{(P-1)}{(M+P-1)}$
- Activation memory： $O(M)$ —— 每个 stage 必须同时缓存 M 个 microbatch 的 activation
- GPipe 不是"更差的 1F1B"，而是**省调度复杂度、换 activation memory**

2.2 1F1B / PipeDream-Flush（业界事实标准）

本质：warmup 填 $(P - i - 1)$ 个 forward，然后 steady state 每次**有 B 做 B，没 B 做 F**，最后 cooldown 清 B。

- Bubble fraction： $\frac{(P-1)}{(M+P-1)}$ —— 和 GPipe 相同
- Activation memory： $O(P)$ —— 每个 stage 最多同时缓存 $(P - i)$ 个 activation
- 这才是 1F1B 对 GPipe 的核心胜利：**不是减 bubble，是减 activation memory**

- 直觉：B 尽早执行 $\rightarrow$ activation 尽早释放

2.3 1F1B 详细时序 (P=4, M=4)

```

t:    0  1  2  3  4  5  6  7  8  9 10 11 12 13
S0:   F0 F1 F2 F3 . . . B0 . B1 . B2 . B3
S1:   . F0 F1 F2 F3 . B0 . B1 . B2 . B3 .
S2:   . . F0 F1 B0 F2 F3 B1 . B2 . B3 . .
S3:   . . . F0 B0 F1 B1 F2 B2 F3 B3 . . .

```

Fi = forward microbatch i Bi = backward microbatch i . = bubble

关键观察：

- **Warmup (t=0..3)**：pipeline 填充。S0 先做 4 个 F，每过一拍下一 stage 加入
- **Steady state (t=3..10)**：F 和 B 交替。中间 stage (S1/S2) 呈现清晰 1F1B pattern；S0/S3 因两端效应不会完全交替
- **Cooldown (t=10..13)**：pipeline 清空。S0 收尾的 **B0·B1·B2·B3** 中间带 bubble，因为要等 B 从尾端逐级传回
- 总时长 $T = 2(M+P-1) = 14$ ；每个 stage 恰好 $2(P-1) = 6$ 格 bubble
- **Bubble fraction** = $(P-1)/(M+P-1) \approx 43\%$ ；M 越大越接近 0，P 越大越差

2.4 Interleaved 1F1B / Virtual Pipeline (Megatron-LM)

每个 physical stage 持有 V 个 virtual chunk，chunk 分散在 layer 空间。

例：P=4, V=2, 总共 24 层 $\rightarrow$ 每 3 层一个 chunk，共 8 chunk：

```

device 0: chunk 0 (L0-2)  + chunk 4 (L12-14)
device 1: chunk 1 (L3-5)  + chunk 5 (L15-17)
device 2: chunk 2 (L6-8)  + chunk 6 (L18-20)
device 3: chunk 3 (L9-11) + chunk 7 (L21-23)

```

- **Bubble fraction**： $(P-1)/(M \cdot V + P-1) \approx$ 原来的 $1/V$
- **代价**：通信量 $\times V$ （每个 microbatch 要在 devices 之间往返 V 次）
- **网络类比**：把一条粗流拆成 V 条细流，用更频繁的小包去填 pipeline 空隙。对 NVLink / 高带宽 IB 合算，弱网不合算

2.5 Zero Bubble (ZB-H1 / ZB-H2)

核心洞察：backward 可以拆成两半：

- **B_input (I)**： dL/dx ，下一 stage 必须等它 $\rightarrow$ 在 critical path 上
- **B_weight (W)**： dL/dW ，只在本地图（accumulate 到 optimizer） $\rightarrow$ 不在 critical path

把 W 延后执行、塞进原本的 bubble 位置：

变体	bubble fraction	activation mem
ZB-H1	$\approx (P-1)/(3M)$ (1F1B 的 $\sim 1/3$)	和 1F1B 相同
ZB-H2	趋近 0	约 2×

ZB 的精妙之处：**bubble 被 compute 填了，不是被通信填了**。这就是为什么论文说 "pipeline parallelism without bubble"。

2.6 DualPipe（DeepSeek-V3）

双向 pipeline：一半 microbatch 从 head 方向流，一半从 tail 反向流，两条流交叉共享 device。

- bubble 近 0
- 需要 **2× model weight** 和约 2× activation
- 适合超大集群的 EP+PP 混合（如 DeepSeek-V3 的 671B MoE）

2.7 对比速查表

方法	bubble fraction	activation mem	comm volume	实现复杂度
GPipe	$(P-1)/(M+P-1)$	$O(M)$	1×	低
1F1B	$(P-1)/(M+P-1)$	$O(P)$	1×	中
Interleaved 1F1B	$(P-1)/(M \cdot V + P - 1)$	$O(P \cdot V)$	V×	中
ZB-H1	$\approx (P-1)/(3M)$	$O(P)$	1×	高
ZB-H2	≈ 0	$\approx 2 \times$	1×	高
DualPipe	≈ 0	$\approx 2 \times \text{model} + 2 \times \text{act}$	$\sim 2 \times$	很高

3. 通信瓶颈：P2P 的那些坑

3.1 NCCL 层的 send/recv

PP 的每一跳是 `ncclSend` + `ncclRecv` 配对，不是 collective。底层两种路径：

- **Intra-node**：GPUDirect P2P over NVLink（或 PCIe P2P，慢得多）
- **Inter-node**：**GPUDirect RDMA**，数据从 GPU HBM 直接走 IB verbs 的 RDMA write，不经过 CPU / system memory

对应你 AWS SIDR 背景：NCCL P2P 大致对应一对 IB QP 之间的 RDMA write，语义接近

`ibv_post_send(IBV_WR_RDMA_WRITE)`。同一对 rank 之间的 P2P stream 是 **FIFO 有序**（类似 TCP 同 flow），不同 channel/stream 可并发。

3.2 PyTorch API

```
python

# 同步
dist.send(tensor, dst=rank)
dist.recv(tensor, src=rank)

# 异步（返回 Work handle）
req = dist.isend(tensor, dst=rank)

# 批量（核心 API，避免 deadlock）
ops = [
    dist.P2POp(dist.isend, send_tensor, next_rank),
    dist.P2POp(dist.irecv, recv_tensor, prev_rank),
]
reqs = dist.batch_isend_irecv(ops)
for r in reqs:
    r.wait()
```

3.3 Deadlock 陷阱（PP 最常见的 bug）

双向 P2P —— 比如 S_i 同时要发 activation 给 S_{i+1} 、又要发 gradient 给 S_{i-1} —— 如果每个 rank 都是 "先 send 后 recv" 的顺序，某些 NCCL 配置下 send buffer 没人取就会阻塞，形成**环形 wait**。

解决方案是 `batch_isend_irecv`：内部把一批 P2P op 包在一对 `ncclGroupStart()` / `ncclGroupEnd()` 里，NCCL runtime 自己排序避免环路。torch/titan 和 PyTorch pipelining 内部所有 P2P 都走这个，**永远不直接写成对 `send` / `recv`**。

网络类比：这和你熟悉的 TCP two-way handshake 死锁一回事 —— 必须有 batched / async 的 buffer 管理。

3.4 重叠优化的三个层次

1. **通信-计算重叠（CUDA stream 层）**：activation send 走 NCCL stream，同一 GPU 的 compute 走 compute stream，两者并发。前提是数据依赖允许
2. **Schedule 层重叠（Interleaved 1F1B）**：每个 stage 有更细的 chunk，每次 send/recv 的 blob 更小，更容易被相邻 compute chunk 掩盖。代价是 V 倍通信次数
3. **把 bubble 填成 compute（ZB）**：这不是通信重叠，是用 **compute 替代 bubble 本身**。
 B_{weight} 没有下游依赖，可以在任意 bubble 位置插入。Action-based scheduler 使得这件事在 runtime 层可表达

3.5 Bubble 之外的其他同步点

- **Optimizer step** : DP 的 all-reduce 必须在 pipeline 清空后才能开始, 这是整个 step 末尾的硬同步点
 - **Grad accumulation + loss scale** : mixed precision 下, 每个 microbatch 贡献一份 grad, 最后要 un-scale 再 all-reduce
 - **VPP 的边界 flush** : interleaved 下最后一个 chunk 的 W 必须在 optimizer 前完成
-

4. torchtitan / PyTorch PP 实现级

4.1 栈层次

```
torchtitan.parallelisms.pipelining_utils ← per-model split helpers
    ↓
torch.distributed.pipelining             ← schedule + runtime
    ↓
torch.distributed (ProcessGroup)         ← P2P primitives
    ↓
NCCL                                     ← transport
```

4.2 核心抽象 : PipelineStage

每个 rank 对应一个 (或多个, interleaved 情形) stage。关键职责 :

- 持有这个 stage 的 `nn.Module` 子集
- 通过 `input_args` / `output_args` 登记 I/O tensor 的 shape + dtype, 在 init 期就 **pre-allocate recv buffer** (避免 per-step malloc)
- 封装 P2P : `_batch_p2p` 统一走 `batch_isend_irecv`
- 暴露 `forward_one_chunk` / `backward_one_chunk` 给 schedule 调用

4.3 Action-based scheduler (PyTorch 2.4+)

```
python

@dataclass
class _Action:
    computation_type: _ComputationType
    # FORWARD | FULL_BACKWARD | BACKWARD_INPUT | BACKWARD_WEIGHT
    # | SEND_F | RECV_F | SEND_B | RECV_B
    microbatch_index: int
    stage_index: int
```

每个 `schedule` 本质是一张 **per-rank 的 Action list**。

`_PipelineScheduleRuntime._step_microbatches` 是 **整个 runtime 的单一入口**，按顺序解释执行：

```
python

for action in self.pipeline_order_with_comms[self.rank]:
    if action.computation_type == _ComputationType.FORWARD:
        self._step_fwd(action)          # 做 forward
    elif action.computation_type == _ComputationType.FULL_BACKWARD:
        self._step_bwd(action)          # full backward
    elif action.computation_type == _ComputationType.BACKWARD_INPUT:
        self._step_bwd_input(action)    # 只算 dL/dx (ZB)
    elif action.computation_type == _ComputationType.BACKWARD_WEIGHT:
        self._step_bwd_weight(action)   # 只算 dL/dW (填 bubble)
    elif action.computation_type == _ComputationType.SEND_F:
        ... # explicit comm op
```

这个抽象让 **ZB / DualPipe** 作为新的 **Action 序列** 加入，而不用重写 runtime。自定义 `schedule` 就是改 **Action list** 的生成函数。

4.4 内置 schedules (`torch/distributed/pipelining/schedules.py`)

- `ScheduleGPipe`, `Schedule1F1B` —— single-stage-per-rank
- `ScheduleInterleaved1F1B`, `ScheduleLoopedBFS`
—— multi-stage-per-rank (interleaved)
- `ScheduleInterleavedZeroBubble`, `ScheduleZBVShape` —— ZB 系列

4.5 torchtitan 的 split 流程

`torchtitan/parallelisms/pipelining_utils.py::pipeline_llama_manual_split` :

1. 按 `pp_size × n_virtual_stages` 把 `model.layers` (`List[TransformerBlock]`) 均匀切
2. 每个 `rank` 拿到自己那段，把其他层替换成 `nn.Identity()` (或删除 + 只保留 `embed/lm_head`)
3. 构造 `PipelineStage(submod, stage_index, num_stages, ..., input_args=example_input)`
4. 用户选 `schedule` : `schedule = Schedule1F1B(stages, n_microbatches, loss_fn)`
5. Training loop : `schedule.step(inputs, target=labels, losses=loss_buffer)`

4.6 Attention residual 与 split point (你正在做的坑)

标准 LLaMA `TransformerBlock` (pre-norm) :



情况 A：block 边界切（torchTitan 默认）

上一 stage 发 x_{out} ，下一 stage 收 x_{in} ，就一个 $[B, S, H]$ tensor。residual 在 block 内已经合并进去，跨 stage 完全不可见。最干净的切法。

情况 B：block 内部切（attention 和 FFN 之间）

- 上一 stage 算到 $x_{mid} = x_{in} + \text{Attn}(\text{Norm}(x_{in}))$ ，把 x_{mid} 发过去
- 如果是单 tensor：跟 block 边界切其实通信量完全一样，因为 residual 已经合进去了
- 如果模型有分离的 residual path（如 MoE gating residual、DenseNet-style cross-layer residual、或你要做的 attention output 作为独立 signal 跨层传）：
 - 必须同时发 x_{mid} 和 attn_out 两个张量 $\rightarrow$ 通信量 $\times 2$
 - autograd 要求 backward 时 residual 的梯度要流回正确的上游。跨 stage 做这件事，要保证 `PipelineStage.output_args` 声明是 tuple，backward 时 `grad_outputs` 也是 tuple

实现层面的建议：

1. 如果只是改标准 pre-norm block 内部，residual 完全在 block 内闭合，切点放哪都只传一个 tensor $\rightarrow$ 不用担心通信
2. 如果引入跨 block 的 attention residual（形如 $x_{n+2} = x_{n+2} + \text{attn_output}_{\{n\}}$ ），residual 必须作为独立 tensor 跨 stage 传：
 - `PipelineStage` 构造时 `output_args = (hidden, attn_residual)` (tuple)
 - P2P buffer 按 tuple size 分配，每跳多一份通信
 - 验证 forward 数值：写一个 single-rank reference run + P2P mock，先保证 forward 对齐再上真 PP
3. 如果 residual 跨多个 stage（比如跳 2 层），需要手动在中间 stage 做 passthrough —— 或者干脆在模型层面重写成 block-local residual

4.7 常见陷阱

- `input_args` / `output_args` 的 shape 和 dtype 必须和真实运行时完全一致。否则 recv buffer 大小不对，NCCL 静默给错数或 assert

- Gradient accumulation : `schedule.step` 内部自动把 M 个 microbatch 的 loss 求和, 但 `loss_fn` 必须是 elementwise reduction (`mean` 要自己除 M)
 - `loss_fn` 只在 **last stage** 被调用; 其他 stage 的 `schedule.step` 要么不传 target, 要么传的 target 会被忽略
 - 用 FSDP + PP 混合时, FSDP 的 all-gather 必须在 PP forward 前完成 —— 这点 torchtitan 已经处理好, 但自己改 split 策略时容易踩
-

5. 推理侧 PP (简述)

训练 PP 的 bubble 可以靠 M 摊薄, **推理 decode 没有这个福利** :

- **Prefill** : 可以 microbatch 化, bubble 行为和训练 forward-only 类似, PP 有收益
 - **Decode** : 每个 token 必须穿越全部 P 个 stage, **每个 token 的 TTBT 被 P 倍放大**。所以生产引擎 (vLLM / TRT-LLM / SGLang) 默认策略是 **TP 吃满 NVLink 域, PP 只在模型放不下时才用** (如跨节点放 MoE expert, 或 405B 这种必须跨 node 的模型)
 - **PD disaggregation 里 PP 的角色** : prefill 实例可以用 PP 拉高 throughput (类似训练 prefill), decode 实例一般不用 PP, 用 TP + 大 batch
-

6. 推荐深入路径

给 torchtitan attention residual 工作的路线

1. 先用 **single-rank + 2-way PP** 跑通 1F1B (`Schedule1F1B`), 用 loss 数值对齐 single-GPU baseline, 验证 split 正确
2. 加 **Interleaved 1F1B** : 改 `Schedule1F1B` → `ScheduleInterleaved1F1B`, `n_virtual_stages=2`, 看 step time 是否如预测下降 $\approx 1/V$
3. 再动 **attention residual 的切点** : 先保 forward 数值, 再验 backward gradient norm, 最后上 convergence 对比

源码阅读优先级

文件 / 函数

```
torch/distributed/pipelining/schedules.py::_PipelineScheduleRuntime._step_microbat
```

`torch/distributed/pipelining/stage.py::_PipelineStageBase._batch_p2p`

`torch/distributed/pipelining/schedules.py::_format_pipeline_order`

`torchtitan/parallelisms/pipelining_utils.py::pipeline_llama_manual_split`

论文 / 资料

- Harlap et al., *PipeDream: Generalized Pipeline Parallelism for DNN Training*, SOSP 2019 —— 1F1B 原始论文
- Narayanan et al., *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM*, SC 2021 —— Interleaved 1F1B 的 Figure 4
- Qi et al., *Zero Bubble Pipeline Parallelism*, ICLR 2024 —— ZB-H1/H2 推导
- DeepSeek-V3 技术报告 §2.3 —— DualPipe 的 Action 布局图，可直接对照

`ScheduleZBVShape`

一个可量化的验证实验

在你当前的 vast.ai 环境（2×RTX 4090，PCIe/TCP）上虽然拿不到真实 IB 的 overlap 行为，但可以验证算法层面的 bubble 公式：

- 固定 $P=2$ ，扫 $M \in \{1, 2, 4, 8, 16\}$
- 测 step time，拟合 $T \approx M \cdot (t_f + t_b) + (P-1) \cdot (t_f + t_b)$
- 看 1F1B vs GPipe 的 step time（应当相同，差异只在 activation memory）
- 看 Interleaved $V=2$ vs $V=1$ 的 step time（有 V 倍通信开销，在 PCIe/TCP 上可能变慢，这正好印证 interleaved 对带宽的依赖）

这些结论在 IB/RoCE 集群上会反转，也正好对应你之前校准过的"consumer GPU 的学习天花板"——算法层看得见，transport 层看不清。